OpenRed - Plataforma Web de Monitorización de Radiación

Resumen Ejecutivo

OpenRed es una plataforma web moderna desarrollada con React y TypeScript para la visualización, análisis y gestión de datos de radiación gamma ambiental recopilados mediante ciencia ciudadana. El sistema permite a usuarios registrados gestionar dispositivos de medición, visualizar datos geoespaciales en tiempo real, y participar en campañas de monitorización colaborativa.

Tecnologías principales: React 19, TypeScript, Material-UI v7, Deck.gl, Mapbox GL, Django REST API

Características destacadas:

- Visualización 3D avanzada de mediciones geoespaciales
- Sistema de autenticación JWT con gestión de sesiones
- Integración con capa MARNA (Mapa de Radiación Natural)
- Escalas de color unificadas y científicamente validadas
- Arquitectura modular y extensible

---

1. Arquitectura del Sistema

1.1 Stack Tecnológico

Componente	Tecnología	Versión	Propósito
Framework	React	19.x	Interfaz de usuario
Lenguaje	TypeScript	5.x	Tipado estático
UI Library	Material-UI	7.x	Componentes visuales
Mapas 3D	Deck.gl	9.x	Visualización geoespacial
Mapas Base	Mapbox GL	2.x	Tiles y navegación
Routing	React Router	6.x	Navegación SPA
HTTP Client	Axios	1.x	Comunicación API
Gestión Estado	React Context API	-	Estado global

1.2 Arquitectura de Componentes

```
┌─────────────────────────────────────────────┐
│           React Application                 │
├─────────────────────────────────────────────┤
│  Context Providers (Auth, Project)          │
├─────────────────────────────────────────────┤
│  Pages                                      │
│  ├── HomePage (Mapa principal)              │
│  ├── StatisticsPage (Estadísticas globales) │
```

```
│  ├── TracksPage (Tracks personales)        │
│  ├── DevicesPage (Gestión dispositivos)    │
│  └── ProfilePage (Perfil usuario)          │
├────────────────────────────────────────────┤
│  Components                                │
│  ├── MapComponent (Deck.gl + Mapbox)       │
│  ├── FiltersPanel                          │
│  ├── Navbar                                │
│  └── ProjectSelector                       │
├────────────────────────────────────────────┤
│  Services                                  │
│  ├── api.ts (Cliente HTTP)                 │
│  └── measurements/ (Servicios específicos) │
├────────────────────────────────────────────┤
│  Utils                                     │
│  └── radiationColors.ts (Escala de colores)│
└────────────────────────────────────────────┘
        ↓
┌────────────────────────────────────────────┐
│      Django REST API Backend               │
└────────────────────────────────────────────┘
```

---

2. Características Principales

2.1 Visualización Geoespacial Avanzada

El componente `MapComponent` integra Deck.gl con Mapbox GL para renderizado de alta performance:

Capacidades técnicas:

- Renderizado GPU de hasta 1M+ puntos simultáneos
- Vista 3D con pitch ajustable (45° por defecto)
- Colores dinámicos basados en dosis de radiación (µSv/h)
- Sistema de zoom adaptativo con agregación H3
- Capa MARNA opcional con esquema TMS

Escala de colores científica:

```
0–0.08 µSv/h    → Azul oscuro a azul claro (Muy bajo)
0.08–0.12 µSv/h → Azul claro a verde (Bajo)
0.12–0.16 µSv/h → Verde a amarillo (Normal)
0.16–0.20 µSv/h → Amarillo a naranja (Moderado)
0.20–0.30 µSv/h → Naranja a rojo (Elevado)
0.30–0.50 µSv/h → Rojo a morado (Alto)
0.50+ µSv/h     → Morado intenso (Muy alto)
```

Implementación centralizada en `src/utils/radiationColors.ts` para consistencia entre todas las páginas.

2.2 Sistema de Autenticación

Flujo de autenticación:

1. Login/Registro → Envío de credenciales al backend Django
2. Backend retorna tokens JWT (access + refresh)
3. Tokens almacenados en localStorage
4. AuthContext gestiona estado de autenticación global
5. Interceptores Axios añaden token a todas las peticiones
6. Refresh automático de tokens expirados

Páginas protegidas:

- DevicesPage (gestión de dispositivos)
- TracksPage (visualización de tracks personales)
- ProfilePage (perfil y configuración)

Páginas públicas:

- HomePage (mapa general)
- StatisticsPage (estadísticas globales)
- LoginPage / RegisterPage

2.3 Gestión de Mediciones

Servicios de mediciones (`src/services/measurements/`):

```
BaseMeasurementService
├── LightPollutionMeasurementService
├── RadiationMeasurementService
└── MeasurementServiceFactory
```

Patrón Factory para extensibilidad hacia nuevos tipos de mediciones (contaminación acústica, calidad del aire, etc.).

Optimizaciones implementadas:

- Agregación por zoom level usando hexágonos H3
- Caché de respuestas API con timestamp
- Debouncing de filtros (500ms)
- Carga diferida de tiles fuera de viewport

2.4 Estadísticas y Analytics

StatisticsPage presenta métricas globales del proyecto:

Métricas mostradas:

- Total de mediciones registradas
- Número de tracks completados
- Contribuidores activos
- Dosis máxima/mínima registrada
- Distancia total recorrida
- Cobertura geográfica (bounding box)
- Equivalencias (vueltas a la Tierra, maratones)

Visualización 3D interactiva:

- Globo terráqueo con capa de países
- Hexágonos H3 coloreados por dosis promedio

- Tooltip con información detallada al hover
- Sin leyendas ni títulos (diseño minimalista)

2.5 Sistema de Proyectos

ProjectContext permite filtrado global por proyecto científico:

```
interface Project {
  id: number;
  name: string;
  description: string;
  active: boolean;
}
```

Selector visible en Navbar con cascada de filtros a todas las páginas con mapas.

---

3. Estructura del Código

3.1 Organización de Directorios

```
openred-react/
├── public/
│   ├── index.html
│   ├── manifest.json
│   └── robots.txt
├── src/
│   ├── components/          # Componentes reutilizables
│   │   ├── MapComponent.tsx
│   │   ├── MapComponent.css
│   │   ├── Navbar.tsx
│   │   ├── FiltersPanel.tsx
│   │   ├── ProjectSelector.tsx
│   │   ├── LoadingProgress.tsx
│   │   └── filters/
│   │       └── DateRangeFilter.tsx
│   ├── pages/               # Páginas principales
│   │   ├── HomePage.tsx
│   │   ├── StatisticsPage.tsx
│   │   ├── TracksPage.tsx
│   │   ├── DevicesPage.tsx
│   │   ├── ProfilePage.tsx
│   │   ├── LoginPage.tsx
│   │   ├── RegisterPage.tsx
│   │   └── ResetPasswordPage.tsx
│   ├── context/             # Context API providers
│   │   ├── AuthContext.tsx
│   │   └── ProjectContext.tsx
│   ├── services/            # Lógica de negocio y API
│   │   ├── api.ts
│   │   ├── mockData.ts
│   │   └── measurements/
```

```
│   │       ├── BaseMeasurementService.ts
│   │       ├── LightPollutionMeasurementService.ts
│   │       ├── RadiationMeasurementService.ts
│   │       ├── MeasurementServiceFactory.ts
│   │       └── index.ts
│   ├── hooks/              # Custom React hooks
│   │   ├── useProjects.ts
│   │   ├── useAdaptiveFilters.ts
│   │   └── useMeasurementsByZoom.ts
│   ├── utils/              # Utilidades y helpers
│   │   └── radiationColors.ts
│   ├── types/              # Definiciones TypeScript
│   │   ├── index.ts
│   │   └── statistics.ts
│   ├── styles/
│   │   └── auth.css
│   ├── theme/
│   │   ├── theme.ts
│   │   └── theme-types.ts
│   ├── App.tsx
│   ├── App.css
│   ├── index.tsx
│   └── index.css
├── build/                  # Build de producción (generado)
├── package.json
├── tsconfig.json
├── README.md
└── .env                    # Variables de entorno (no versionado)
```

3.2 Componentes Clave

MapComponent.tsx

Componente principal de visualización geoespacial.

Props:

```tsx
interface MapComponentProps {
  measurements?: Measurement[];
  loading?: boolean;
  filters?: FilterState;
  onFilterChange?: (filters: FilterState) => void;
}
```

Características:

- DeckGL con múltiples capas (ScatterplotLayer, H3HexagonLayer)
- Controles interactivos (filtros, zoom, MARNA)
- Leyenda dinámica con gradientes CSS
- Responsive con offsets calculados dinámicamente

Navbar.tsx

Barra de navegación con estado de autenticación.

Funcionalidad:

- Logo y nombre del proyecto
- Botón "Estadísticas" (público)
- Selector de proyectos (si hay múltiples)
- Menú de usuario autenticado:
  - Dispositivos
  - Tracks
  - Perfil
  - Logout

FiltersPanel.tsx

Panel de filtros con estados sincronizados.

Filtros disponibles:

- Rango de fechas (DateRangeFilter)
- Proyectos (si aplicable)
- Dispositivos
- Rango de dosis

3.3 Servicios y API

api.ts

Cliente HTTP centralizado con interceptores.

```
const api = axios.create({
  baseURL: process.env.REACT_APP_API_URL || 'http://localhost:8000',
  headers: {
    'Content-Type': 'application/json',
  },
});

// Interceptor: añadir token JWT
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('access_token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

// Interceptor: refresh token automático
api.interceptors.response.use(
  (response) => response,
  async (error) => {
    if (error.response?.status === 401) {
      // Lógica de refresh token
    }
    return Promise.reject(error);
```

```
  }
);
```

Endpoints principales:

Endpoint	Método	Descripción
`/api/auth/login/`	POST	Autenticación
`/api/auth/register/`	POST	Registro de usuario
`/api/users/me/`	GET	Perfil usuario
`/api/devices/`	GET/POST	Gestión dispositivos
`/api/measurements/`	GET	Mediciones con filtros
`/api/measurements/by-zoom/`	GET	Mediciones agregadas
`/api/tracks/`	GET	Tracks de usuario
`/api/statistics/global/`	GET	Estadísticas globales
`/api/statistics/heatmap/`	GET	Datos para heatmap H3
`/api/projects/`	GET	Lista de proyectos

MeasurementServiceFactory

Patrón Factory para diferentes tipos de mediciones:

```
export class MeasurementServiceFactory {
  static getService(type: string): BaseMeasurementService {
    switch (type) {
      case 'radiation':
        return new RadiationMeasurementService();
      case 'light_pollution':
        return new LightPollutionMeasurementService();
      default:
        return new RadiationMeasurementService();
    }
  }
}
```

---

4. Configuración e Instalación

4.1 Requisitos del Sistema

Software necesario:

- Node.js >= 16.x
- npm >= 8.x
- Git

Servicios externos:

- Backend Django REST API (OpenRed backend)
- Mapbox API Token (gratuito hasta 50k vistas/mes)
- Servidor de tiles MARNA (opcional): https://map.open-red.es/marna_tiles/

4.2 Instalación Paso a Paso

1. Clonar repositorio:

```
git clone https://github.com/frasanz/openred-react.git
cd openred-react
```

2. Instalar dependencias:

```
npm install
```

3. Configurar variables de entorno:

Crear archivo `.env` en la raíz:

```
# Backend API
REACT_APP_API_URL=https://api.open-red.es
# o para desarrollo local:
# REACT_APP_API_URL=http://localhost:8000

# Mapbox Token (obligatorio)
REACT_APP_MAPBOX_TOKEN=pk.eyJ1IjoieW91cnVzZXJuYW1lIiwiYSI6InlvdXJ0b2tlbiJ9...

# Opcional: habilitar modo debug
REACT_APP_DEBUG=true
```

4. Obtener Mapbox Token:

- Registrarse en https://www.mapbox.com/
- Ir a Account → Tokens
- Crear nuevo token con permisos de lectura
- Copiar token al `.env`

5. Iniciar servidor de desarrollo:

```
npm start
```

Abre http://localhost:3000 en el navegador.

4.3 Build para Producción

```
# Crear build optimizado
npm run build
```

```
# Output en carpeta build/
# Archivos estáticos listos para deploy
```

Configuración de servidor web (ejemplo Nginx):

```
server {
    listen 80;
    server_name app.open-red.es;

    root /var/www/openred-react/build;
    index index.html;

    # Redirigir todas las rutas a index.html (SPA)
    location / {
        try_files $uri $uri/ /index.html;
    }

    # Cache de assets estáticos
    location /static/ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }

    # Proxy a backend API
    location /api/ {
        proxy_pass http://localhost:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

4.4 Variables de Entorno

Variable	Requerida	Descripción	Default
`REACT_APP_API_URL`	Sí	URL del backend Django	`http://localhost:8000`
`REACT_APP_MAPBOX_TOKEN`	Sí	Token de Mapbox API	-
`REACT_APP_DEBUG`	No	Habilitar logs debug	`false`
`REACT_APP_ENABLE_MARNA`	No	Habilitar capa MARNA	`true`

---

5. Guía de Desarrollo

5.1 Convenciones de Código

TypeScript:

- Interfaces con prefijo `I` cuando sea ambiguo
- Types para uniones y alias
- Props con sufijo `Props`

- Tipado explícito en funciones públicas

React:

- Functional components con hooks
- Props desestructurados
- useCallback para funciones pasadas como props
- useMemo para cálculos costosos

Estilos:

- Material-UI `sx` prop para estilos inline
- CSS modules para estilos complejos
- Variables del theme para colores

Naming:

- Componentes: PascalCase (`MapComponent.tsx`)
- Hooks: camelCase con prefijo `use` (`useProjects.ts`)
- Utils: camelCase (`radiationColors.ts`)
- Constants: UPPER_SNAKE_CASE

5.2 Agregar Nuevo Tipo de Medición

Ejemplo: Contaminación Acústica

1. **Crear servicio:**

```
// src/services/measurements/NoiseMeasurementService.ts
import { BaseMeasurementService } from './BaseMeasurementService';

export class NoiseMeasurementService extends BaseMeasurementService {
  protected getEndpoint(): string {
    return '/api/measurements/noise/';
  }

  protected transformData(data: any): any {
    return {
      ...data,
      noise_level_db: data.value,
    };
  }
}
```

2. **Registrar en Factory:**

```
// src/services/measurements/MeasurementServiceFactory.ts
case 'noise':
  return new NoiseMeasurementService();
```

3. **Añadir escala de colores:**

```
// src/utils/noiseColors.ts
export const getNoiseColor = (decibels: number): [number, number, number] => {
  if (decibels < 40) return [0, 255, 0]; // Verde (silencioso)
  // ... más rangos
};
```

4. **Usar en componentes:**

```
const service = MeasurementServiceFactory.getService('noise');
const measurements = await service.getMeasurements(filters);
```

5.3 Testing

```
# Ejecutar tests
npm test

# Coverage report
npm test -- --coverage

# Tests en modo watch
npm test -- --watch
```

Estructura de tests:

```
src/
├── components/
│   ├── MapComponent.tsx
│   └── MapComponent.test.tsx
├── services/
│   ├── api.ts
│   └── api.test.ts
└── utils/
    ├── radiationColors.ts
    └── radiationColors.test.ts
```

5.4 Debugging

React DevTools:

- Inspeccionar árbol de componentes
- Ver props y state en tiempo real
- Profiler para performance

Redux DevTools (si se añade Redux):

- Time-travel debugging
- Inspección de acciones

Mapbox/Deck.gl Debug:

```
// En MapComponent.tsx
const [viewState, setViewState] = useState({
  // ... config
  debug: true, // Activa logs de Deck.gl
});
```

Network debugging:

```
// En api.ts
api.interceptors.request.use((config) => {
  console.log('Request:', config.method, config.url, config.data);
  return config;
});
```

---

6. Despliegue

6.1 Entornos

Desarrollo:

- URL: http://localhost:3000
- Backend: http://localhost:8000
- Hot reload habilitado

Staging:

- URL: https://staging.open-red.es
- Backend: https://api-staging.open-red.es
- Build optimizado sin sourcemaps

Producción:

- URL: https://app.open-red.es
- Backend: https://api.open-red.es
- Build optimizado + minificado + compresión

6.2 Deployment con Docker

Dockerfile:

```
# Stage 1: Build
FROM node:16-alpine AS build

WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production

COPY . .
RUN npm run build

# Stage 2: Serve
FROM nginx:alpine
```

```
COPY --from=build /app/build /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf

EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

docker-compose.yml:

```
version: '3.8'

services:
  frontend:
    build: .
    ports:
      - "80:80"
    environment:
      - REACT_APP_API_URL=https://api.open-red.es
      - REACT_APP_MAPBOX_TOKEN=${MAPBOX_TOKEN}
    depends_on:
      - backend
```

Desplegar:

```
docker-compose up -d
```

6.3 CI/CD con GitHub Actions

.github/workflows/deploy.yml:

```
name: Deploy to Production

on:
  push:
    branches: [main]

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v3

      - name: Setup Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '16'
          cache: 'npm'

      - name: Install dependencies
```

```
        run: npm ci

      - name: Run tests
        run: npm test -- --coverage

      - name: Build
        run: npm run build
        env:
          REACT_APP_API_URL: ${{ secrets.API_URL }}
          REACT_APP_MAPBOX_TOKEN: ${{ secrets.MAPBOX_TOKEN }}

      - name: Deploy to server
        uses: appleboy/scp-action@master
        with:
          host: ${{ secrets.HOST }}
          username: ${{ secrets.USERNAME }}
          key: ${{ secrets.SSH_KEY }}
          source: "build/*"
          target: "/var/www/openred-react/"
```

---

7. Optimización y Performance

7.1 Técnicas Implementadas

Code Splitting:

```
// Lazy loading de páginas
const StatisticsPage = lazy(() => import('./pages/StatisticsPage'));
const TracksPage = lazy(() => import('./pages/TracksPage'));

// Suspense boundary
<Suspense fallback={<LoadingProgress />}>
  <Routes>
    <Route path="/statistics" element={<StatisticsPage />} />
  </Routes>
</Suspense>
```

Memoization:

```
// Evitar re-renders innecesarios
const mapLayers = useMemo(() => [
  new ScatterplotLayer({
    id: 'measurements',
    data: measurements,
    // ... config
  }),
], [measurements]);

const handleFilter = useCallback((filters) => {
```

```
  // ... lógica
}, [dependencies]);
```

Virtualización:

- Deck.gl maneja virtualización de puntos automáticamente
- Solo renderiza elementos en viewport

Bundle optimization:

```
// package.json
{
  "build": "react-scripts build",
  "postbuild": "npm run compress",
  "compress": "gzip -9 -k -r build/static/"
}
```

7.2 Métricas de Performance

Objetivos:

- First Contentful Paint (FCP): < 1.5s
- Largest Contentful Paint (LCP): < 2.5s
- Time to Interactive (TTI): < 3.5s
- Cumulative Layout Shift (CLS): < 0.1

Herramientas de medición:

```
# Lighthouse
npm install -g lighthouse
lighthouse http://localhost:3000 --view

# Bundle analyzer
npm install --save-dev webpack-bundle-analyzer
npm run build -- --stats
npx webpack-bundle-analyzer build/bundle-stats.json
```

7.3 Optimización de Imágenes

Formatos recomendados:

- Logos: SVG
- Fotos: WebP (fallback JPEG)
- Iconos: SVG inline o icon fonts

Lazy loading:

```
<img
  src={thumbnailUrl}
  loading="lazy"
  alt="Device"
/>
```

---

8. Seguridad

8.1 Medidas Implementadas

1. Autenticación segura:

- Tokens JWT con expiración
- Refresh tokens en httpOnly cookies (si backend soporta)
- Logout limpia tokens completamente

2. Protección XSS:

- React escapa HTML por defecto
- Sanitización de inputs con DOMPurify (si se añade)
- CSP headers en producción

3. Protección CSRF:

- Tokens CSRF en formularios
- SameSite cookies

4. HTTPS obligatorio:

- Redirect HTTP → HTTPS
- HSTS headers
- Secure cookies

5. Validación de inputs:

- Yup schemas para formularios
- Validación cliente + servidor

8.2 Variables Sensibles

NUNCA commitear:

- `.env` con tokens reales
- Credenciales de API
- Claves privadas

Gestión segura:

```
# .gitignore
.env
.env.local
.env.production
```

En producción:

```
# Variables de entorno en servidor
export REACT_APP_API_URL=https://api.open-red.es
export REACT_APP_MAPBOX_TOKEN=pk.real_token_here
```

8.3 Auditoría de Dependencias

```
# Escanear vulnerabilidades
npm audit

# Fix automático
npm audit fix

# Fix de breaking changes
npm audit fix --force
```

---

9. Resolución de Problemas

9.1 Problemas Comunes

❌ **"Mapbox token inválido"**

```
# Verificar token en .env
echo $REACT_APP_MAPBOX_TOKEN

# Reiniciar servidor de desarrollo
npm start
```

❌ **"CORS error al llamar API"**

Backend debe permitir origen del frontend:

```
# Django settings.py
CORS_ALLOWED_ORIGINS = [
    "http://localhost:3000",
    "https://app.open-red.es",
]
```

❌ **"Module not found"**

```
# Limpiar node_modules
rm -rf node_modules package-lock.json
npm install
```

❌ **"Deck.gl no renderiza"**

Verificar WebGL en navegador:

```
https://get.webgl.org/
```

❌ **"Build falla"**

```
# Aumentar memoria de Node.js
export NODE_OPTIONS=--max_old_space_size=4096
npm run build
```

9.2 Logs de Debug

Habilitar logs verbose:

```
// src/index.tsx
if (process.env.REACT_APP_DEBUG === 'true') {
  // Log todas las peticiones
  api.interceptors.request.use(config => {
    console.log('🔵 Request:', config);
    return config;
  });

  // Log todas las respuestas
  api.interceptors.response.use(response => {
    console.log('🟢 Response:', response);
    return response;
  });
}
```

9.3 Recursos de Ayuda

Documentación oficial:

- React: https://react.dev/
- Deck.gl: https://deck.gl/
- Mapbox GL: https://docs.mapbox.com/mapbox-gl-js/
- Material-UI: https://mui.com/

Comunidad:

- GitHub Issues: https://github.com/frasanz/openred-react/issues
- Stack Overflow: tag `openred`

---

10. Hoja de Ruta

10.1 Completado ✅

- [x] Arquitectura base React + TypeScript
- [x] Sistema de autenticación JWT
- [x] Visualización 3D con Deck.gl + Mapbox
- [x] Integración capa MARNA
- [x] Escala de colores unificada
- [x] Página de estadísticas globales
- [x] Gestión de dispositivos
- [x] Visualización de tracks personales
- [x] Filtros avanzados con fecha y proyecto
- [x] Responsive design
- [x] Optimización de performance

10.2 En Desarrollo 🚧

- [ ] Internacionalización (i18n) español/inglés
- [ ] Progressive Web App (PWA)
- [ ] Exportación de datos (CSV, GeoJSON)
- [ ] Gráficas temporales de mediciones
- [ ] Comparación de tracks

10.3 Planificado 📋

- [ ] Modo offline con Service Workers
- [ ] Notificaciones push
- [ ] Dashboard de analytics avanzado
- [ ] Integración con más tipos de sensores
- [ ] API GraphQL (alternativa a REST)
- [ ] Tests E2E con Cypress
- [ ] Storybook para documentación de componentes
- [ ] Tema oscuro (dark mode)

---

11. Contribución

11.1 Cómo Contribuir

1. Fork y clone:

```
git clone https://github.com/tu-usuario/openred-react.git
cd openred-react
git remote add upstream https://github.com/frasanz/openred-react.git
```

2. Crear rama feature:

```
git checkout -b feature/nueva-funcionalidad
```

3. Desarrollar y testear:

```
npm test
npm run build
```

4. Commit con convención:

```
git commit -m "feat: añadir exportación de datos a CSV"
```

Tipos de commit:

- `feat` : Nueva funcionalidad
- `fix` : Corrección de bug
- `docs` : Cambios en documentación
- `style` : Formato, sin cambios de código
- `refactor` : Refactorización de código
- `test` : Añadir o modificar tests

- `chore` : Tareas de mantenimiento

5. Push y Pull Request:

```
git push origin feature/nueva-funcionalidad
```

Crear PR en GitHub con descripción detallada.

11.2 Code Review

Checklist:

- [ ] Tests pasan (`npm test`)
- [ ] Build exitoso (`npm run build`)
- [ ] Código formateado (Prettier/ESLint)
- [ ] Sin warnings en consola
- [ ] Documentación actualizada
- [ ] Commits siguen convención

11.3 Código de Conducta

- Respetar a todos los contribuidores
- Feedback constructivo y profesional
- No discriminación
- Privacidad y seguridad primero

---

12. Licencia

Este proyecto está desarrollado para el Proyecto OpenRed.

Contacto:

- Web: https://open-red.es
- Email: info@open-red.es
- GitHub: https://github.com/frasanz/openred-react

---

13. Anexos

13.1 Glosario

Término	Definición
µSv/h	Microsieverts por hora, unidad de dosis de radiación
H3	Sistema de indexación geoespacial hexagonal de Uber
JWT	JSON Web Token, estándar para autenticación
TMS	Tile Map Service, esquema de coordenadas para tiles
MARNA	Mapa de Radiación Natural, capa base de radiación de fondo
SPA	Single Page Application

Context API	Sistema de gestión de estado global de React
Factory Pattern	Patrón de diseño para creación de objetos

13.2 Referencias

1. Documentación React: https://react.dev/
2. TypeScript Handbook: https://www.typescriptlang.org/docs/
3. Material-UI v7: https://mui.com/material-ui/
4. Deck.gl Documentation: https://deck.gl/docs
5. Mapbox GL JS API: https://docs.mapbox.com/mapbox-gl-js/
6. Uber H3: https://h3geo.org/
7. JWT Introduction: https://jwt.io/introduction
8. React Router v6: https://reactrouter.com/
9. Axios Documentation: https://axios-http.com/
10. Create React App: https://create-react-app.dev/

13.3 Changelog

Ver archivo `CHANGELOG.md` para historial detallado de cambios por versión.

Última actualización: Noviembre 2025
Versión de documentación: 1.0

Generado para el Proyecto OpenRed - Plataforma de Monitorización de Radiación mediante Ciencia Ciudadana

🛠️ Tecnologías

- **React 19** con Hooks y Context API
- **TypeScript** para tipado estático
- **Material-UI v7** para componentes UI
- **React Router Dom** para navegación
- **Deck.gl** y **Mapbox** para visualización avanzada de mapas 3D
- **Axios** para llamadas API
- **React Hook Form** con validación Yup
- **Styled Components** para estilos personalizados

📁 Estructura del proyecto

```
src/
├── components/          # Componentes reutilizables
│   ├── MapComponent.tsx    # Componente de mapa con Deck.gl y Mapbox
│   └── Navbar.tsx          # Barra de navegación
├── pages/              # Páginas principales
│   ├── HomePage.tsx        # Página principal con mapa
│   ├── LoginPage.tsx       # Página de inicio de sesión
│   ├── RegisterPage.tsx    # Página de registro
│   ├── DevicesPage.tsx     # Gestión de dispositivos
│   ├── ProfilePage.tsx     # Perfil de usuario
│   └── CampaignsPage.tsx   # Campañas y misiones
├── services/           # Servicios API
```

```
│   └── api.ts            # Cliente Axios y endpoints
├── context/            # Context API
│   └── AuthContext.tsx   # Contexto de autenticación
├── types/              # Definiciones TypeScript
│   └── index.ts          # Interfaces principales
├── hooks/              # Custom hooks
├── utils/              # Utilidades
└── App.tsx             # Componente principal
```

🔧 Configuración

Variables de entorno

Crea un archivo `.env` basado en `.env.example`:

```
cp .env.example .env
```

Variables disponibles:

- `REACT_APP_API_URL`: URL del backend Django (por defecto: http://localhost:8000)
- `REACT_APP_MAPBOX_TOKEN`: Token de acceso de Mapbox (requerido para los mapas)

Instalación

```
# Instalar dependencias
npm install

# Iniciar en modo desarrollo
npm start

# Construir para producción
npm run build
```

Configuración de Mapbox

Para habilitar la visualización de mapas, necesitas obtener un token de acceso de Mapbox:

1. Ve a Mapbox y crea una cuenta gratuita
2. Accede a tu panel de control y obtén tu token de acceso
3. Añade el token a tu archivo `.env`:

```
REACT_APP_MAPBOX_TOKEN=tu_token_de_mapbox_aqui
```

> ***Nota**: Sin el token de Mapbox, los mapas no se mostrarán correctamente. El mapa utiliza Deck.gl para renderizado 3D avanzado con Mapbox como capa base.*

🌐 Conexión con el Backend

El frontend se conecta al backend Django a través de API REST:

- **Autenticación**: `/api/auth/login/` , `/api/auth/register/`
- **Usuarios**: `/api/users/me/` , `/api/profile/`
- **Dispositivos**: `/api/devices/` , `/api/device-models/`
- **Mediciones**: `/api/measurements/`
- **Campañas**: `/api/campaigns/`
- **Misiones**: `/api/missions/`

📱 Funcionalidades principales

Autenticación

- Login y registro de usuarios
- Gestión de tokens JWT
- Rutas protegidas
- Persistencia de sesión

Mapa interactivo (Deck.gl + Mapbox)

- Visualización avanzada 3D de mediciones de radiación gamma
- Renderizado de alta performance con Deck.gl
- Colores dinámicos basados en intensidad de radiación (gradiente verde-rojo)
- Filtros interactivos por fecha, intensidad y campaña
- Tooltips informativos al hacer hover sobre los puntos
- Vista 3D con pitch y bearing ajustables
- Navegación fluida con controles táctiles

Gestión de dispositivos

- Lista de dispositivos del usuario
- Creación y edición de dispositivos
- Asociación con modelos de dispositivos
- Estados activo/inactivo

Perfil de usuario

- Edición de datos personales
- Estadísticas de actividad
- Estado de la cuenta

Campañas y misiones

- Creación y gestión de campañas
- Misiones asociadas a campañas
- Estados temporales (próxima, activa, finalizada)
- Gestión de participantes

🎨 Diseño y UX

- **Material Design**: Seguimiento de las guías de Material-UI
- **Tema personalizado**: Colores corporativos y tipografía
- **Responsive**: Adaptado a diferentes tamaños de pantalla
- **Accesibilidad**: Cumple estándares WCAG
- **Feedback visual**: Loading states, mensajes de error/éxito

🔄 Migración desde Django

Este frontend React reemplaza las templates de Django manteniendo:

- Misma funcionalidad que el frontend Django original
- Mejora en experiencia de usuario
- API REST como puente entre ambos sistemas
- Posibilidad de coexistencia durante la transición

📚 Scripts disponibles

`npm start`

Inicia el servidor de desarrollo en http://localhost:3000

`npm test`

Ejecuta los tests en modo watch

`npm run build`

Construye la aplicación para producción en la carpeta `build`

`npm run eject`

Nota: esta operación es irreversible

🤝 Contribución

1. Fork el proyecto
2. Crea una rama para tu feature (`git checkout -b feature/AmazingFeature`)
3. Commit tus cambios (`git commit -m 'Add some AmazingFeature'`)
4. Push a la rama (`git push origin feature/AmazingFeature`)
5. Abre un Pull Request

📄 Licencia

Este proyecto está bajo la licencia correspondiente a OpenRed.

🐛 Issues conocidos

- Los date pickers requieren configuración adicional de locales para español
- Algunos endpoints del backend pueden requerir ajustes para coincidir con la API esperada
- La geolocalización requiere HTTPS en producción

🔮 Próximas mejoras

- [ ] Internacionalización (i18n)
- [ ] Progressive Web App (PWA)
- [ ] Modo offline
- [ ] Notificaciones push
- [ ] Dashboard de analytics
- [ ] Exportación de datos
- [ ] Integración con más proveedores de mapas

`npm run build`

Builds the app for production to the `build` folder.
It correctly bundles React in production mode and optimizes the build for the best performance.

The build is minified and the filenames include the hashes.
Your app is ready to be deployed!

See the section about deployment for more information.

`npm run eject`

Note: this is a one-way operation. Once you `eject`, you can't go back!

If you aren't satisfied with the build tool and configuration choices, you can `eject` at any time. This command will remove the single build dependency from your project.

Instead, it will copy all the configuration files and the transitive dependencies (webpack, Babel, ESLint, etc) right into your project so you have full control over them. All of the commands except `eject` will still work, but they will point to the copied scripts so you can tweak them. At this point you're on your own.

You don't have to ever use `eject`. The curated feature set is suitable for small and middle deployments, and you shouldn't feel obligated to use this feature. However we understand that this tool wouldn't be useful if you couldn't customize it when you are ready for it.

Learn More

You can learn more in the Create React App documentation.

To learn React, check out the React documentation.